ROLAND System: Precision UAV Landing

Autonomous Navigation and Robotics Localization

LONG Yiqi, WANG Xuanting, HU Junye
Department of Aeronautical and Aviation Engineering
The Hong Kong Polytechnic University

Outline

- >Background
- >Literature reviewer
- >Methodology
- >Experimental validation
- >Conclusion
- >Future work

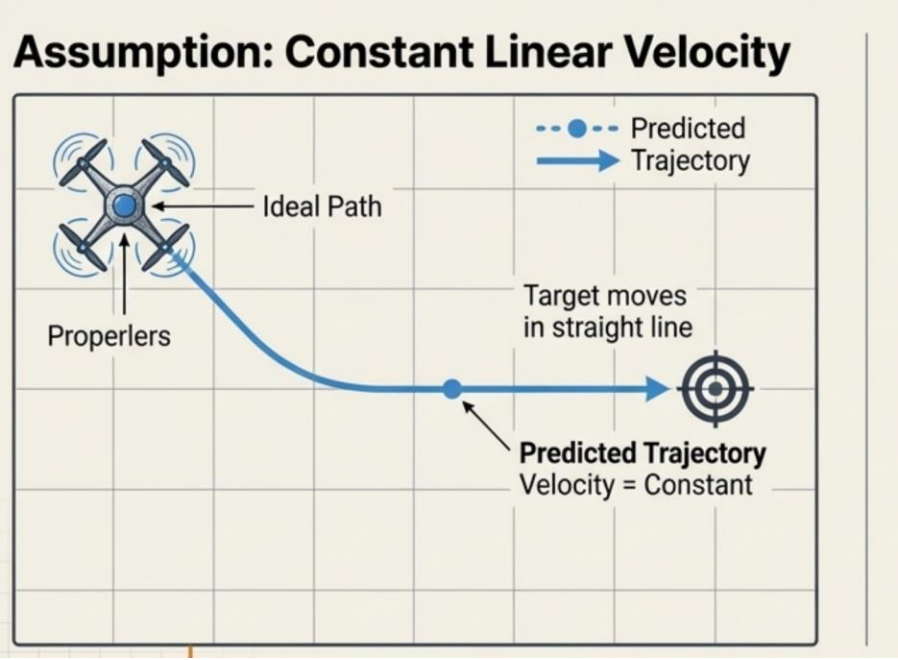
01. BACKGROUND

Reinforcing Pain Points & Industrial Demands

LIMITATIONS OF "GREENHOUSE" RESEARCH

Motion Restrictions

Most algorithms assume target platforms are static or move in linear paths. Prediction models often collapse during sudden acceleration or turning maneuvers.

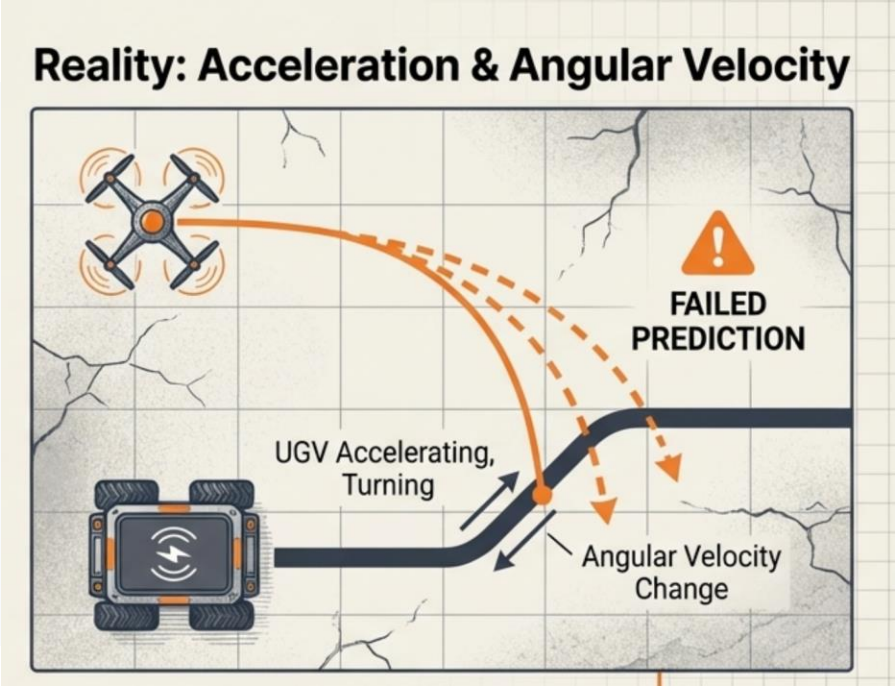


LIMITATIONS OF "GREENHOUSE" RESEARCH

Motion Restrictions

Most algorithms assume target platforms are static or move in linear paths. Prediction models often collapse during sudden acceleration or turning maneuvers.

Most predictive tracking models immediately fail when the target platform accelerates, brakes, or change vectors. In the real world, targets are uncooperative.



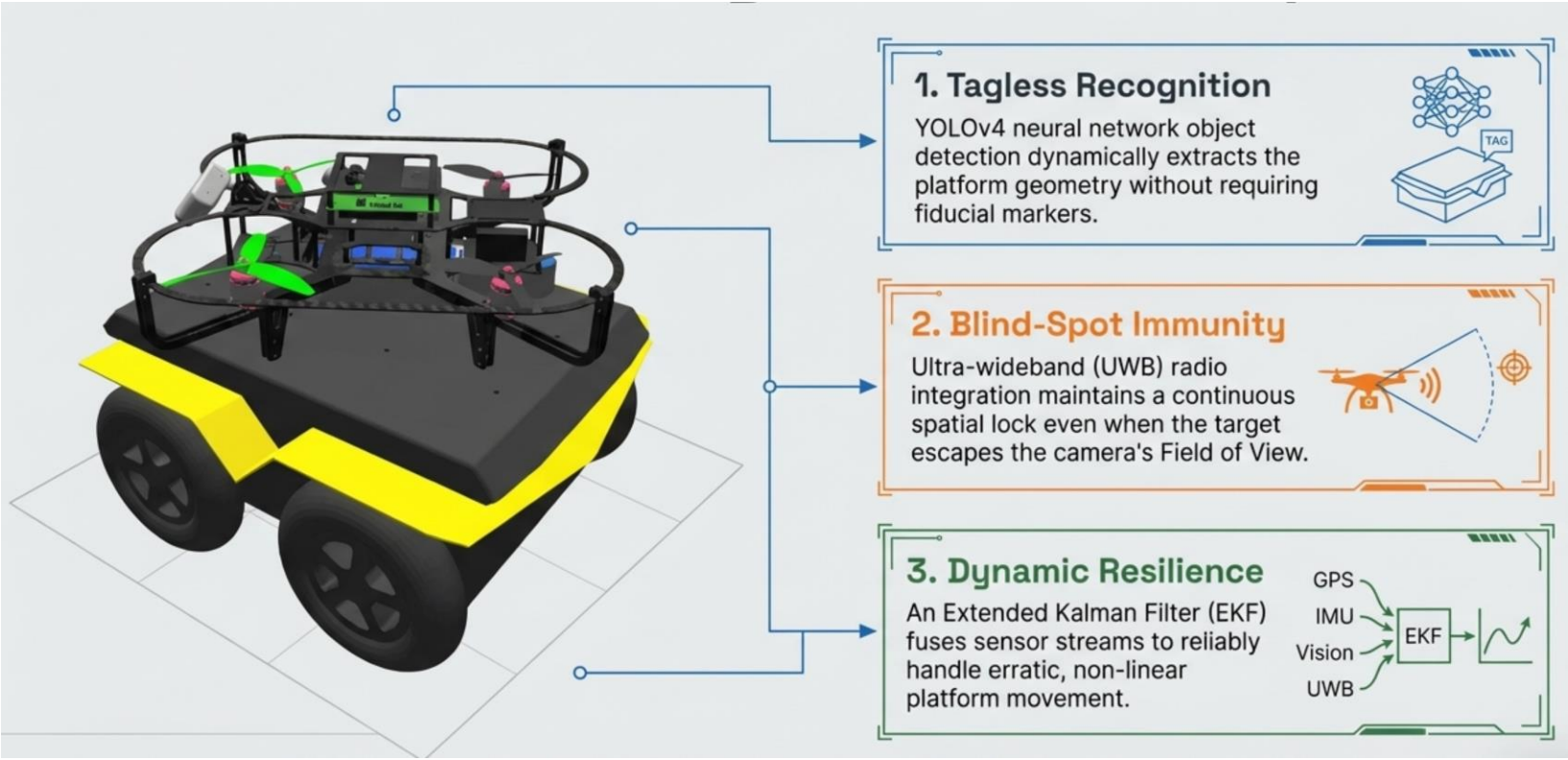
SENSOR FAILURE & "GAPS" RISKS

Environmental Dependency

Current tech relies heavily on specific markers (QR code tags). These require a constant line-of-sight, failing in unstructured or messy environments.

The Marker Dependency: Physical Occlusion

Enter ROLAND: Landing Without Assumptions



A Web of Interlocking Redundancy

Real-World Hazard	RGB-D Vision	UWB Radio	Odometry (Encoders)
Environmental Occlusion	✗	✓	✓
Close-Proximity FoV Loss	✗	✓	✓
Cumulative Drift & Scale Error	✓	✓	✗
High-Resolution Targeting	✓	✗ (Range Only)	✗

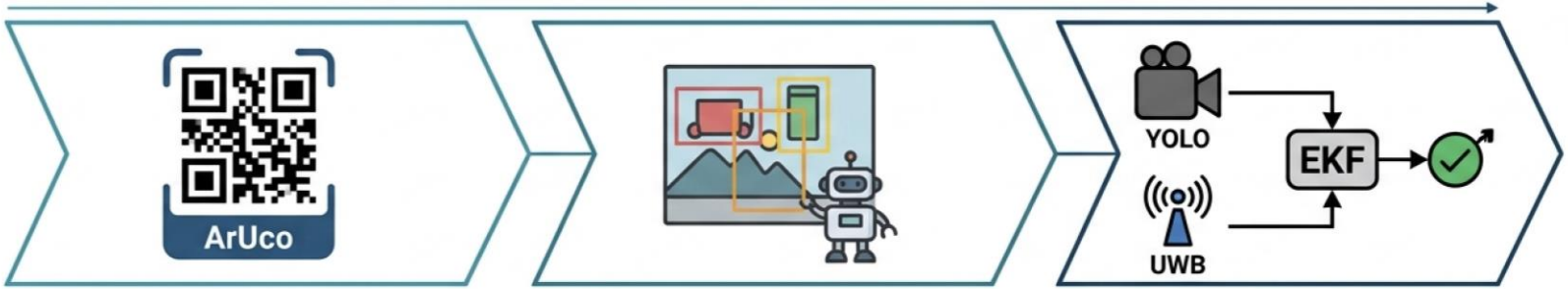
No single sensor survives the real world. ROLAND weaves their overlapping strengths to create an inescapable safety net for target acquisition.

02. Literature Review

The Evolution of Autonomous Landing

Literature Review: Beyond constraints

The Label Era | The Object Detection Era | The Robust Era



Stage 1: Fiducial Markers

- **Reference**: Garrido-Jurado et al. (ArUco/QR)
- **Limitation**: Field of View (FoV) dependency & susceptibility to lighting

Stage 2: Deep Learning

- **Reference**: Redmon et al. (YOLO Framework)
- **Limitation**: Depth estimation drift and occlusion issues.

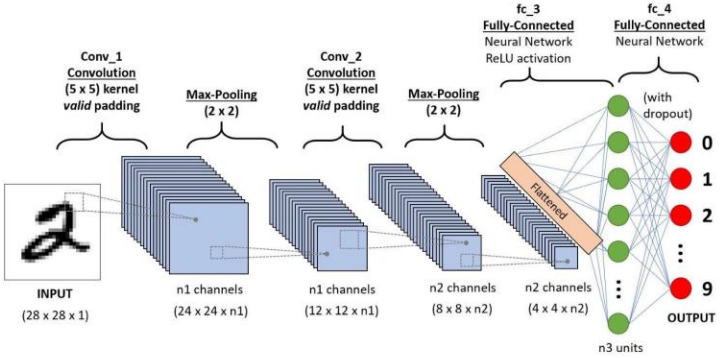
Stage 3: Multi-Sensor Fusion

- **Innovation**: ROLAND Framework (Kim et al., 2021)
- **Solution**: Combining Vision (YOLO) with UWB (Ultra-Wideband) via Extended Kalman Filter (EKF).

YOLO & CNN: The vision backbone

- CNN (Convolutional Neural Network): The "Brain" that extracts hierarchical features from raw pixels.

- YOLO: Object detection CNN. A state-of-art model that analyzes images and identifies objects with bounding boxes, class labels and confidence scores.



3.Methodology

The process of training AI

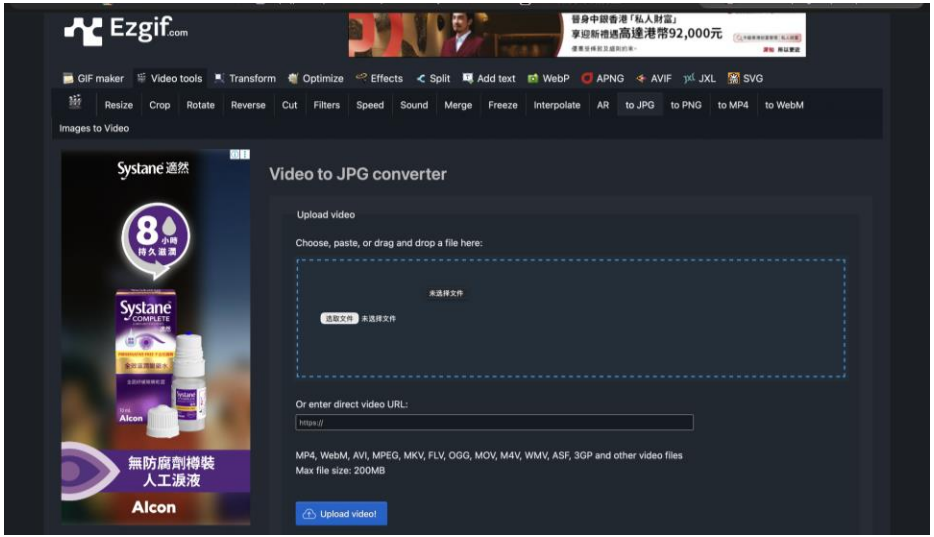
data collection

Gazebo,Rviz

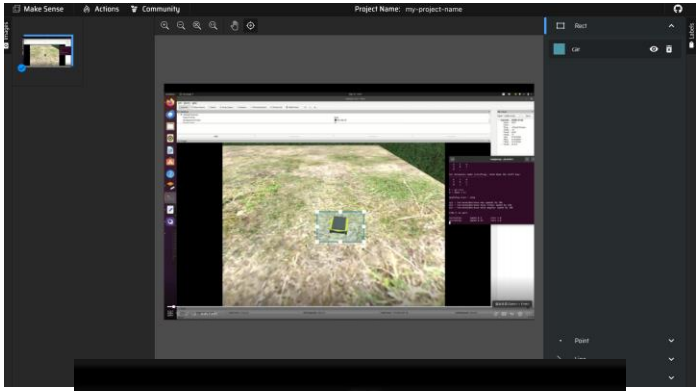
https://youtu.be/HdqZ7y4VsfE

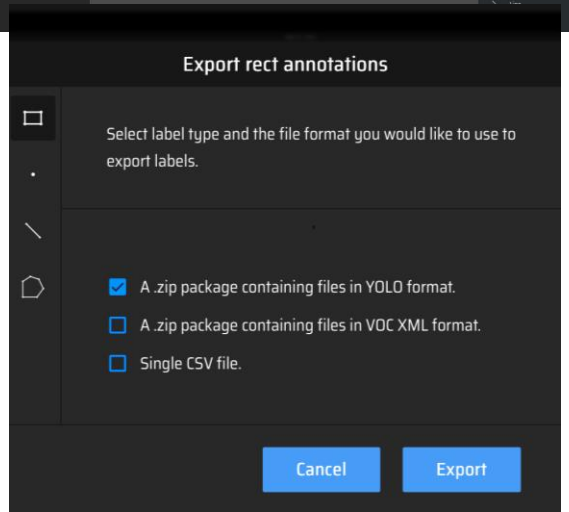
https://youtu.be/ttiLf1V08EE

Extract photos from the video (data preparation)



Data annotation





- Import the pictures you have extracted.
- Use the mouse to draw a frame around the "car" in the picture.
- Name this frame with a label, for example, "car".

Export format must select YOLO format!!

```
my_dataset/
├── images/
│   ├── train/    # Store the training set images
│   └── val/      # Store the images of the validation set
└── labels/
    ├── train/    # The TXT file corresponding to the training set is stored
    └── val/      # The TXT file corresponding to the validation set is stored
└── auto/     #configuration file
```

run.py.txt training_p

文件 编辑 查看

5 0.726496 0.791643 0.018993 0.030389

- Category
- Center Point X
- Center Point Y
- Width
- Height

auto.yaml

C: > Users > xuanting > Desktop > Gazebo_car_YOLO > Gazebo_car_YOLO > dataset > auto.yaml

```yaml
# Auto-generated
path: C:\Users\xuanting\Desktop\Gazebo_car_YOLO\Gazebo_car_YOLO\dataset
train: images/train
val: images/val
names:
  0: worker
  1: no hard hat
  2: no safety vest
  3: no hard hat no safety vest
  4: Heavy Machinery
  5: Car
  6: Truck
```

Prepare the environment (RTX5060)

> Anaconda (conda 12.8)

> Python (Create virtual environment - Activate environment）

> PyTorch2.12.0.dev20260328+cu128 (Responsible for all tensor calculations and the operation of neural networks)

> YOLO Official Library (Ultralytics)

Start training the model

Confirm that the path in your auto file is correct.

auto.yaml

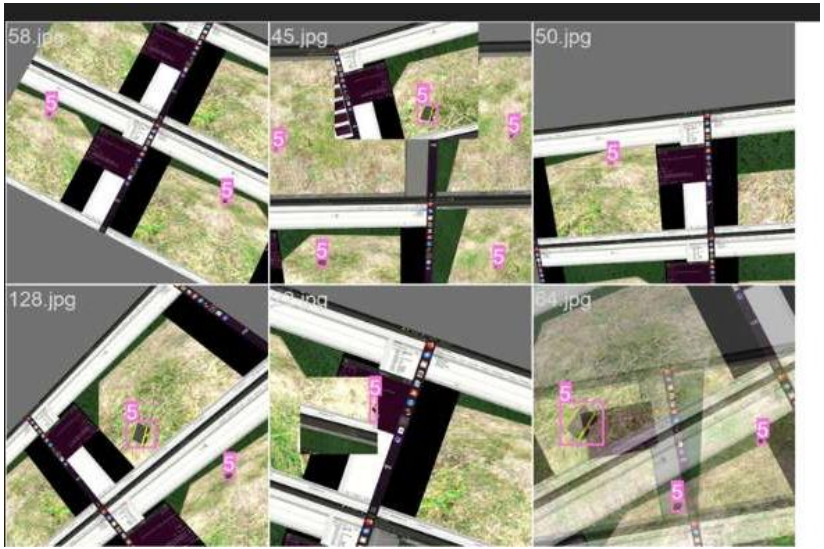
C: > Users > xuanting > Desktop > Gazebo_car_YOLO > Gazebo_car_YOLO > dataset > auto.yaml

```yaml
# Auto-generated
path: C:\Users\xuanting\Desktop\Gazebo_car_YOLO\Gazebo_car_YOLO\dataset
train: images/train
val: images/val
names:
  0: worker
  1: no hard hat
  2: no safety vest
  3: no hard hat no safety vest
  4: Heavy Machinery
  5: Car
  6: Truck
```

名称	修改日期	类型	大小
dataset	2026/3/31 0:19	文件夹	
gazebo_car	2026/3/31 0:19	文件夹	
train	2026/3/31 0:19	Python 源文件	10 KB
vedieo1	2026/3/31 0:19	MP4 文件	111,988 KB
vedieo3	2026/3/31 0:19	MP4 文件	10,028 KB
yolo26n.pt	2026/3/31 0:19	PT 文件	5,415 KB

train.py

```
"epochs": 150,
"imgsz": 1280,
"batch": 10,
```




```
# Geometric
"degrees": 45.0,      # image rotation (+/- deg)
"translate": 0.2,   # image translation (+/- fraction)
"scale": 0.3,       # image scale (+/- gain)
"shear": 0.0,        # image shear (+/- deg)
"perspective": 0.0002, # perspective (+/- fraction)
```

run

> Cd C:\Users\xuanting\Desktop\Gazebo_car_YOLO\Gazebo_car_YOLO

> activate yolo_env

> python train.py --data dataset/auto.yaml --model yolo26n.pt

```
                 from  n    params  module                                       arguments
  0                 -1  1       464  ultralytics.nn.modules.conv.Conv             [3, 16, 3, 2]
  1                 -1  1      4672  ultralytics.nn.modules.conv.Conv             [16, 32, 3, 2]
  2                 -1  1      6640  ultralytics.nn.modules.block.C3k2            [32, 64, 1, False, 0.25]
  3                 -1  1     36992  ultralytics.nn.modules.conv.Conv             [64, 64, 3, 2]
  4                 -1  1     26080  ultralytics.nn.modules.block.C3k2            [64, 128, 1, False, 0.25]
  5                 -1  1    147712  ultralytics.nn.modules.conv.Conv             [128, 128, 3, 2]
  6                 -1  1     87040  ultralytics.nn.modules.block.C3k2            [128, 128, 1, True]
  7                 -1  1    295424  ultralytics.nn.modules.conv.Conv             [128, 256, 3, 2]
  8                 -1  1    346112  ultralytics.nn.modules.block.C3k2            [256, 256, 1, True]
  9                 -1  1    164608  ultralytics.nn.modules.block.SPPF            [256, 256, 5, 3, True]
 10                 -1  1    249728  ultralytics.nn.modules.block.C2PSA           [256, 256, 1]
 11                 -1  1         0  torch.nn.modules.upsampling.Upsample         [None, 2, 'nearest']
 12            [-1, 6]  1         0  ultralytics.nn.modules.conv.Concat           [1]
 13                 -1  1    119808  ultralytics.nn.modules.block.C3k2            [384, 128, 1, True]
 14                 -1  1         0  torch.nn.modules.upsampling.Upsample         [None, 2, 'nearest']
 15            [-1, 4]  1         0  ultralytics.nn.modules.conv.Concat           [1]
 16                 -1  1     34304  ultralytics.nn.modules.block.C3k2            [256, 64, 1, True]
 17                 -1  1     36992  ultralytics.nn.modules.conv.Conv             [64, 64, 3, 2]
 18           [-1, 13]  1         0  ultralytics.nn.modules.conv.Concat           [1]
 19                 -1  1     95232  ultralytics.nn.modules.block.C3k2            [192, 128, 1, True]
 20                 -1  1    147712  ultralytics.nn.modules.conv.Conv             [128, 128, 3, 2]
 21           [-1, 10]  1         0  ultralytics.nn.modules.conv.Concat           [1]
 22                 -1  1    463104  ultralytics.nn.modules.block.C3k2            [384, 256, 1, True, 0.5, True]
 23       [16, 19, 22]  1    243906  ultralytics.nn.modules.head.Detect           [7, 1, True, [64, 128, 256]]
YOLO26n summary: 260 layers, 2,506,530 parameters, 2,506,530 gradients, 5.8 GFLOPs
```

CNN

- Each row (0 to 23): Represents a "processing workshop" in the AI brain (a layer of the neural network).
- Module (module name): For example, Conv (convolution layer, used to extract basic features such as edges and colors), C3k2, SPPF, etc.
- Params (number of parameters): How many adjustable "wheels" does this layer have.

```
AMP: checks passed
train: Fast image access  (ping: 0.00.0 ms, read: 400.7156.1 MB/s, size: 229.0 KB)
train: Scanning C:\Users\xuanting\Desktop\Gazebo_car_YOLO\Gazebo_car_YOLO\dataset\labels\train\videos.cache... 137 images, 6 backgrounds, 0 corrupt: 100% ━━━━━━━━━━━━ 143/143  0.0
s
val: Fast image access  (ping: 0.10.1 ms, read: 250.859.7 MB/s, size: 283.1 KB)
val: Scanning C:\Users\xuanting\Desktop\Gazebo_car_YOLO\Gazebo_car_YOLO\dataset\labels\val.cache... 13 images, 1 backgrounds, 0 corrupt: 100% ━━━━━━━━━━━━ 14/14  0.0s
optimizer: MuSGD(lr=0.01, momentum=0.937) with parameter groups 114 weight(decay=0.0), 126 weight(decay=0.0005), 126 bias(decay=0.0)
```

- **`train: Fast image access (ping: 0.00.0 ms, read: 400.7156.1 MB/s, size: 229.0 KB)`**
  Test of image reading speed for training set.

- **`train: Scanning ...\train\videos.cache... 137 images, 6 backgrounds, 0 corrupt: 100%`**
  Scan the training set labels and generate the cache

- **`val: Fast image access ...`**
  `val: Scanning ...\val.cache... 13 images, 1 backgrounds, 0 corrupt: 100%`
  Reading test and scanning of the validation set.

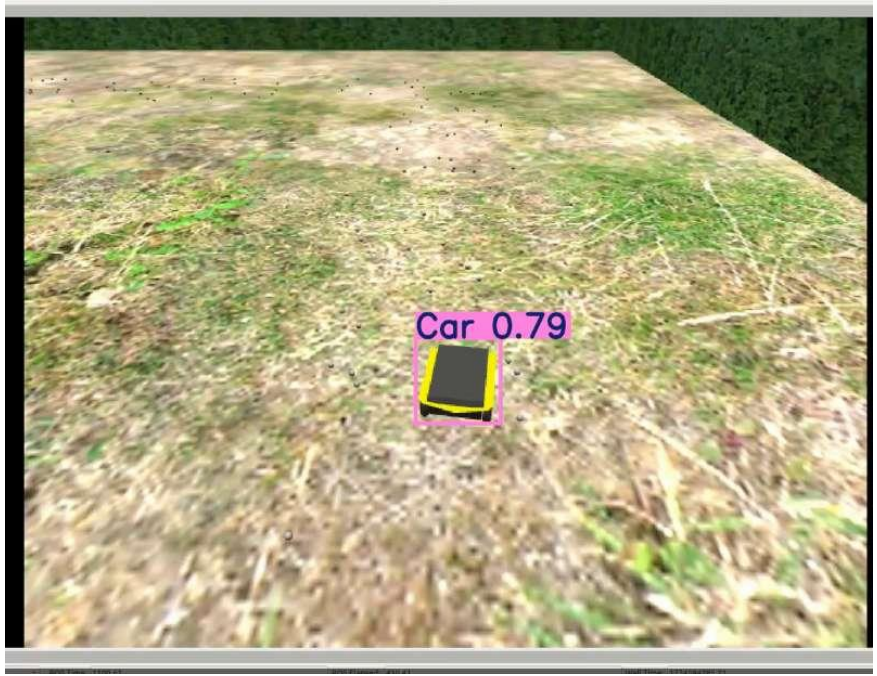
```
Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
1/150      0.744G      1.813      9.314   0.005872          6        640: 100% ━━━━━━━━━━━━ 36/36 2.0it/s 17.9s
           Class     Images  Instances     Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 2/2 1.5s/it 3.0s
             all         14         13          0          0          0          0

Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
2/150      0.744G      1.628      7.771   0.005306          6        640: 100% ━━━━━━━━━━━━ 36/36 8.9it/s 4.0s
           Class     Images  Instances     Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 2/2 23.0it/s 0.1s
             all         14         13     0.0135      0.308      0.172     0.0231

Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
3/150      0.744G      1.472      5.729   0.004907          5        640: 100% ━━━━━━━━━━━━ 36/36 10.3it/s 3.5s
           Class     Images  Instances     Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 2/2 20.6it/s 0.1s
             all         14         13      0.313      0.154      0.189     0.0526

Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
4/150      0.744G      1.566      5.568   0.005223          3        640: 100% ━━━━━━━━━━━━ 36/36 10.4it/s 3.5s
           Class     Images  Instances     Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 2/2 22.0it/s 0.1s
             all         14         13     0.0563      0.462     0.0519      0.021

Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
5/150      0.744G      1.428      5.221   0.004331          9        640: 100% ━━━━━━━━━━━━ 36/36 10.9it/s 3.3s
           Class     Images  Instances     Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 2/2 20.0it/s 0.1s
             all         14         13      0.199      0.154      0.136     0.0296

Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
6/150      0.744G       1.56       5.36   0.005039         10        640: 100% ━━━━━━━━━━━━ 36/36 10.4it/s 3.5s
           Class     Images  Instances     Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 2/2 22.1it/s 0.1s
             all         14         13      0.627      0.462      0.585      0.148

Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
7/150      0.746G      1.437      4.784   0.004602         14        640: 58% ━━━━━━────── 21/36 7.1it/s 2.1s<2.1s
```

- GPU_mem (0.744G): Memory usage of the GPU.
- box_loss: Error of bounding boxes.
- cls_loss: Error of classification.
- dfl_loss: Error of distribution focus.
- Instances: The current batch of images contains several targets.

4.Experimental validation

We can confirm whether an ideal result will be achieved by recording a new video and submitting it to our trained AI model.

https://youtu.be/PPOhmMO90x8



05. Conclusion

Success Replication & Validation

Conclusion

Successful porting of the original research into a standardized simulation environment.

- **Key takeaways:**
  - Validated YOLOv5 real-time performance with detection accuracy.
  - Understood the underlying architecture of ROLAND system with EKF.
  - Reflected on Environment Parity (ROS/Gazebo compatibility challenge)

6. FUTURE WORK

Academic Foresight & Algorithm Evolution

STATE VECTOR AUGMENTATION

Heading-Aware EKF

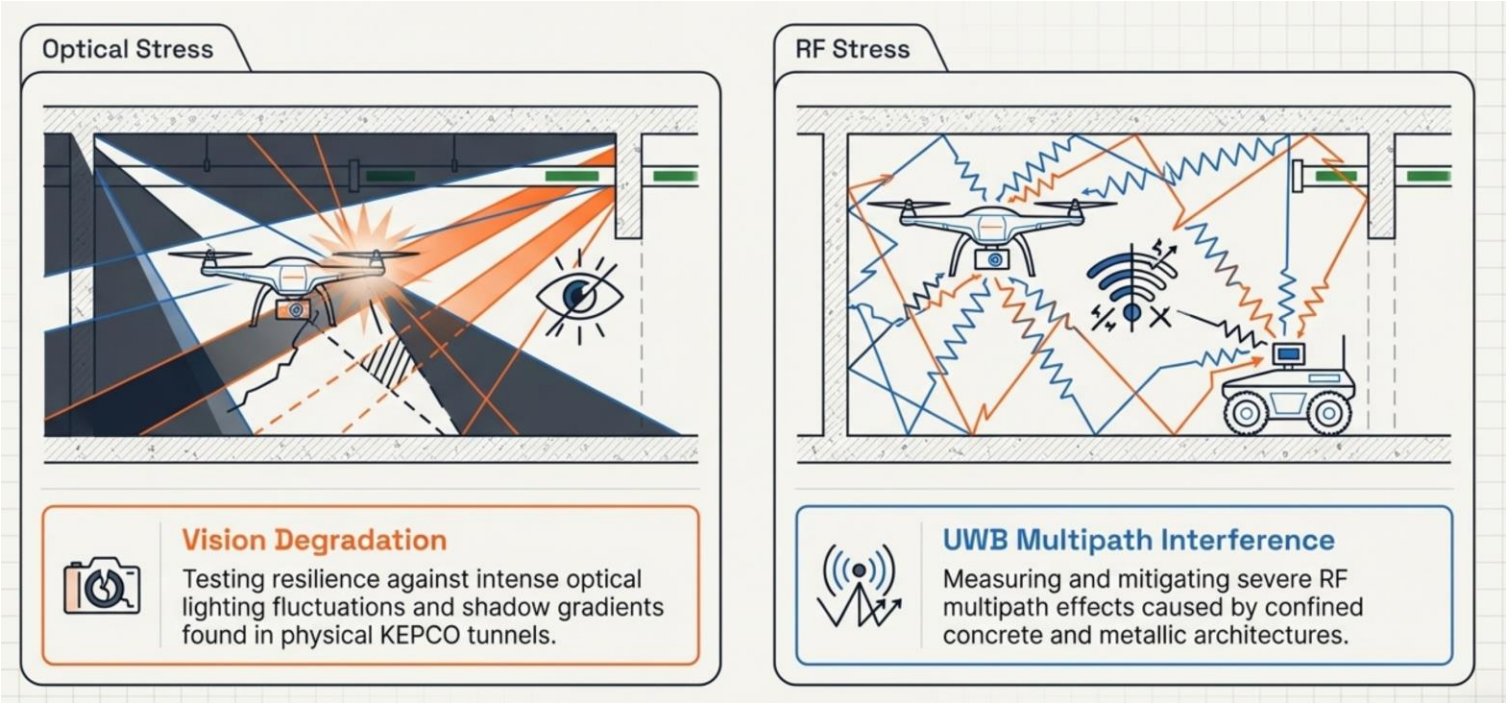
Current EKF models primarily estimate relative position. Our next step is to integrate **Relative Heading** into the state vector.

$$x_k = [\, x, y, z, \dot{x}, \dot{y}, \dot{z}, \psi_{rel} \,]^T$$

Landing faster and precisely

The next algorithmic frontier is incorporating Relative Heading into the state vector. This enables the UAV to pre-align its yaw with the platform's directional vector-critical for docking on narrow, constrained mobile charging pads.

SIM-TO-REAL TRANSFORMATION



Thank you for listening!

References

Falanga, D., Zanchettin, A., Simovic, A., Delmerico, J., & Scaramuzza, D. (2017). Vision-based autonomous quadrotor landing on a moving platform. *IEEE/RSJ International Symposium on Safety, Security and Rescue Robotics (SSRR)*, 200–207. https://doi.org/10.1109/ssrr.2017.8088164

Garrido-Jurado, S., Muñoz-Salinas, R., Madrid-Cuevas, F., & Marín-Jiménez, M. (2014). Automatic generation and detection of highly reliable fiducial markers under occlusion. Pattern Recognition, 47(6), 2280–2292. https://doi.org/10.1016/j.patcog.2014.01.005

Kim, C., Lee, E. M., Choi, J., Jeon, J., Kim, S., & Myung, H. (2021). ROLAND: Robust Landing of UAV on Moving Platform using Object Detection and UWB based Extended Kalman Filter. 2021 21st International Conference on Control, Automation and Systems (ICCAS), 249–254. https://doi.org/10.23919/iccas52745.2021.9649920